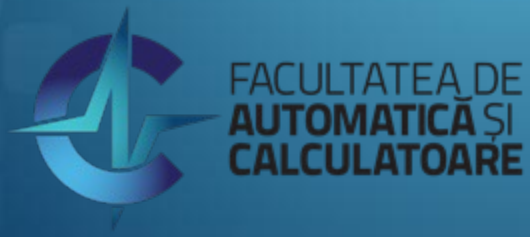


Algoritmi Paraleli și Distribuiți
MPI

Radu-Ioan Ciobanu
radu.ciobanu@upb.ro





Reminder

- **NU** se face curs miercurea viitoare (30.11)
- **NU** se fac laboratoare săptămâna viitoare (28.11 – 02.12)

Feedback

https://forms.office.com/r/MxvEiuLFFW



MPI - *Message Passing Interface*

- **MPI** este un **standard** pentru comunicarea prin mesaje elaborat de *MPI Forum*.
- Sisteme anterioare:
  - Intel (NX/2) Express
  - nCUBE (Vertex)
  - PARMACS
  - Zipcode
  - Chimp
  - PVM
  - Chameleon
  - PICL
- MPI are la bază modelul proceselor comunicante prin mesaje.

MPI - *Message Passing Interface*

- MPI este o (*specificație* pentru o) **bibliotecă**, nu un limbaj.
- MPI specifică reguli de apel pentru:
  - C / C++
  - FORTRAN 77 / FORTRAN 90
- Istoric:
  - **MPI 1** (1993 – SC93) – orientat pe comunicarea punct la punct
  - **MPI 2** (1997) include:
    - Procese dinamice
    - Comunicarea *one-sided*
    - Operații de I/E paralele

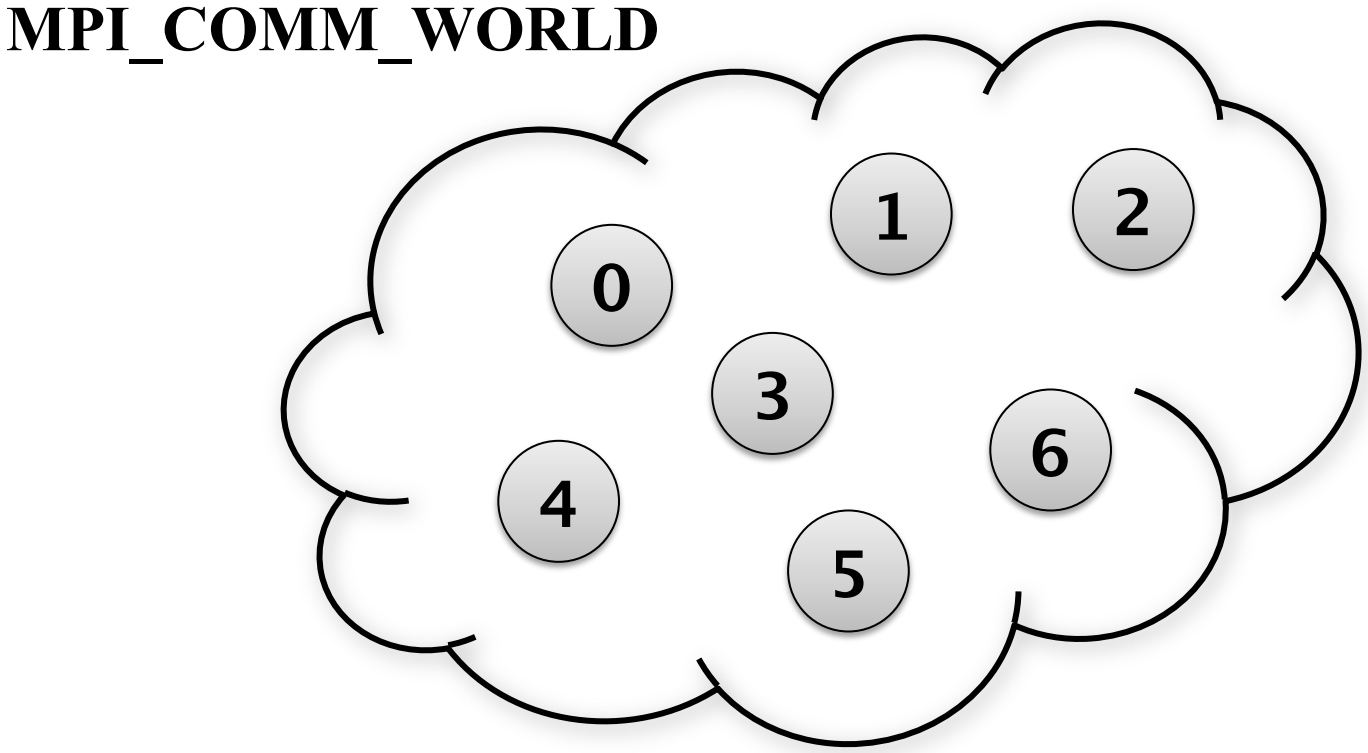
Obiective MPI

- Standardizare
  - Singura bibliotecă de transmitere de mesaje ce poate fi considerată standard
- Portabilitate
  - Codul nu trebuie modificat la portarea aplicațiilor
- Performanță
  - Implementarile producătorilor pot exploata carcateristici hardware native
- Funcționalitate
  - Peste 115 rutine doar în **MPI 1**
- Disponibilitate
  - Implementări atât publice, cât și de la producători

Elemente noi introduse de MPI

- **Tampon de comunicație** – definit de tripleta:
  - *(adresă, contor, tip_de_date)*
- **Context**
  - Fiecare comunicare de mesaj se derulează într-un *context*.
  - Mesajele sunt întotdeauna primite în contextul în care au fost transmise.
  - Mesajele transmise în contexte diferite nu interferă.
  - Mesajele sunt caracterizate de:
    - **context**
    - **tag**
- **Grup de procese**
  - Contextul este partajat de un *grup de procese.*
  - Fiecare proces are un rang (număr întreg).
  - Deci, fiecare proces e caracterizat de:
    - **grup**
    - **rang în grup**

Elemente noi introduse de MPI



Există un comunicator predefinit:

MPI_COMM_WORLD

Structura generală a unui program MPI

```
1   #include <mpi.h>
2
3   .
4   .
5   .
6   /* Initialize MPI Environment */
7   int MPI_Init(int *argc, char ***argv);
8   int MPI_Initialized(int *flag);
9   .
10  .
11  .
12  /* Do work and make message passing calls */
13  int MPI_Comm_rank(MPI_Comm comm, int *rank);
14  int MPI_Comm_size(MPI_Comm comm, int *size);
15  .
16  .
17  .
18  /* Terminate MPI Environment */
19  int MPI_Finalize();
```

Comunicarea punct la punct – operații de bază

- Forma clasică:
  - send (adresă, lungime, destinație, tag)
  - recv (adresă, lungime_maximă, sursă, tag, lungime_efectivă)
- În MPI:
  - MPI_SEND (buf, count, datatype, dest, tag, comm)
  - MPI_RECV (buf, count, datatype, source, tag, comm, status)
- Limbajul C:

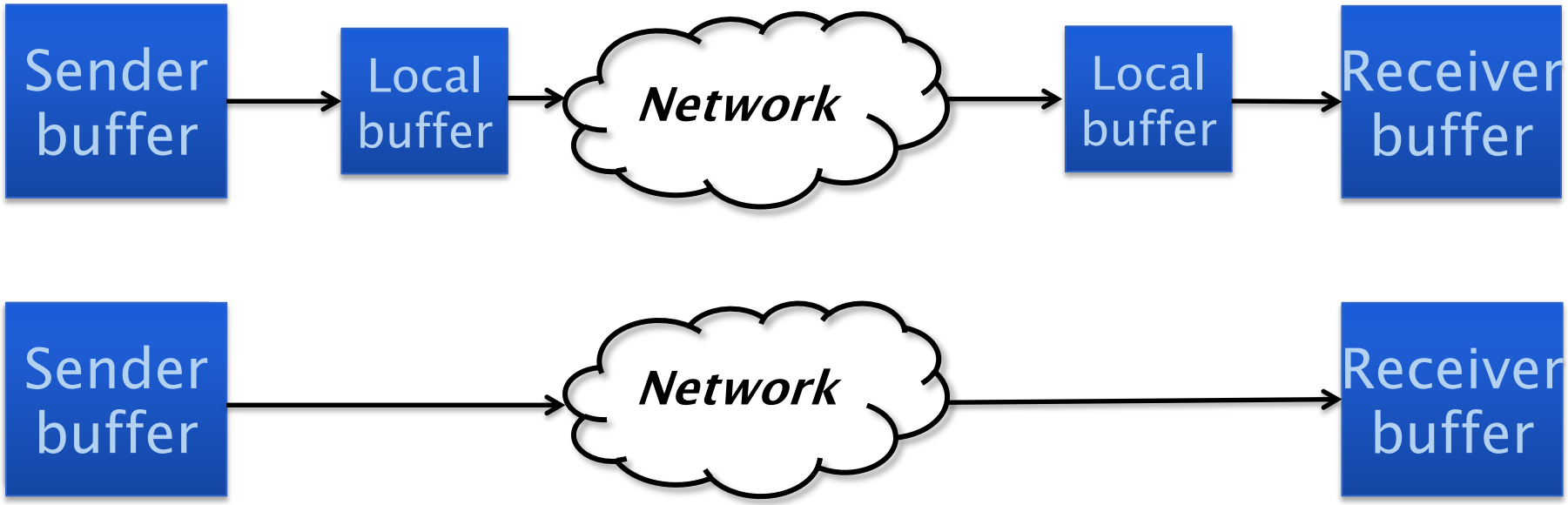
```
int MPI_Send (void *buf, int count,
              MPI_Datatype datatype, int dest,
              int tag, MPI_Comm comm)
int MPI_Recv (void *buf, int count,
               MPI_Datatype datatype, int source,
               int tag, MPI_Comm comm, MPI_Status *status)
```

Exemplu

```
1  #include "mpi.h"
2
3  int main (int argc, char **argv)
4  {
5      char message[40];
6      int myrank;
7      MPI_Status status;
8
9      MPI_Init(&argc, &argv);
10     MPI_Comm_rank (MPI_COMM_WORLD, &myrank);
11     if (myrank == 0) {
12         strcpy(message, Hello, there);
13         MPI_Send(message, strlen(message), MPI_CHAR, 1, 99, MPI_COMM_WORLD);
14     } else {
15         MPI_Recv(message, 20, MPI_CHAR, 0, 99, MPI_COMM_WORLD, &status);
16         printf("\tReceived:%s \n", message);
17     }
18
19     MPI_Finalize();
20     return 0;
21 }
```

Moduri de comunicare

- Apelul *receive* este **blocant**
- Moduri de comunicare:
  - cu buffer local
  - fără buffer local

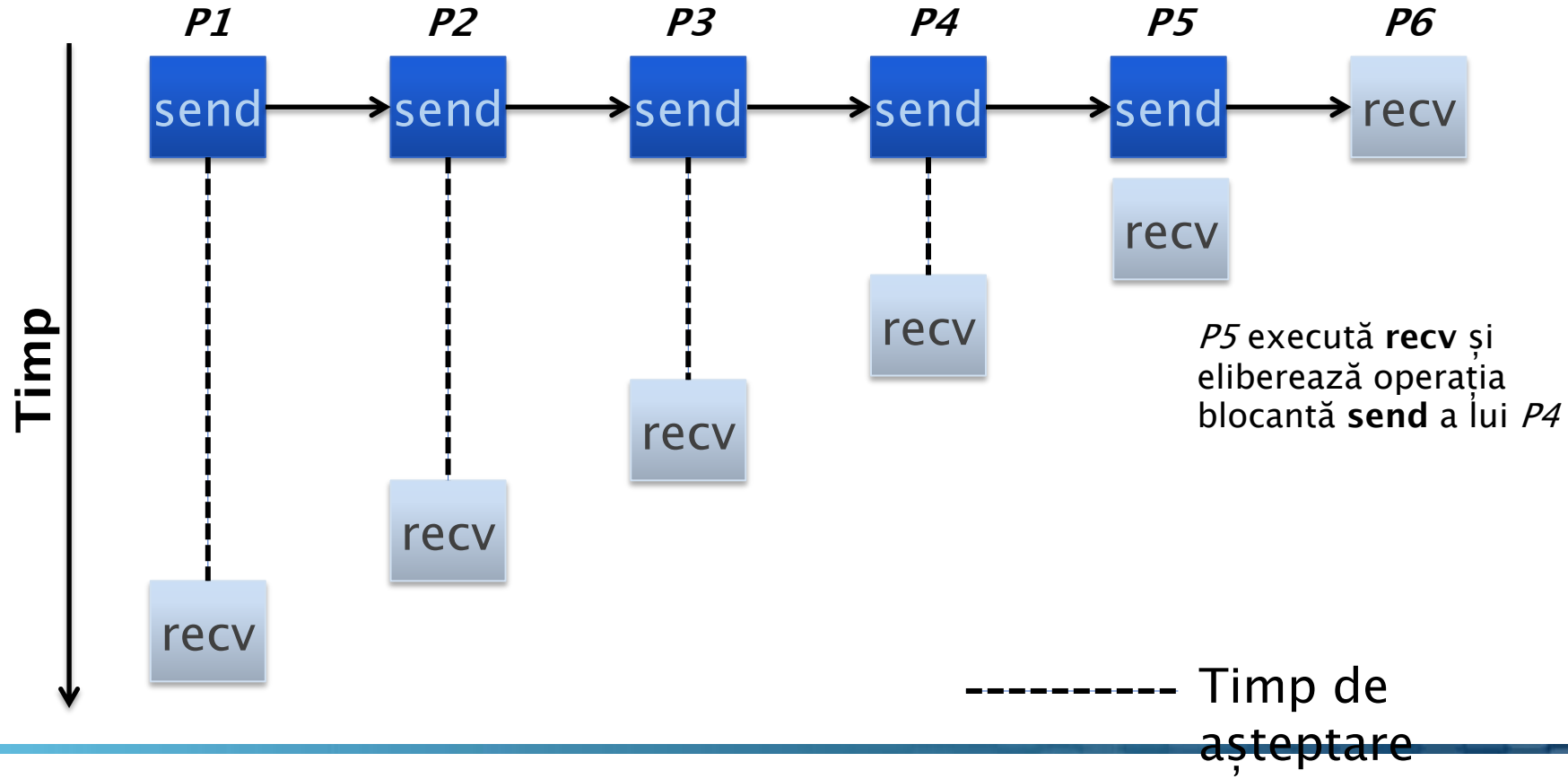


Modurile de comunicaţie MPI

Tip	Condiția de terminare
Synchronous send	Finalizare atunci când operația de recepție a fost terminată cu succes.
Buffered send	Se termină întotdeauna (cu exceptia apariției unei erori), indiferent de operația de recepție.
Standard send	Trimiterea mesajului (fără a se cunoaște starea operatiei de recepție).
Ready send	Se termină întotdeauna (cu excepția apariției unei erori), însă folosirea e condiționată de apelul anterior la destinație a operației de recepție.

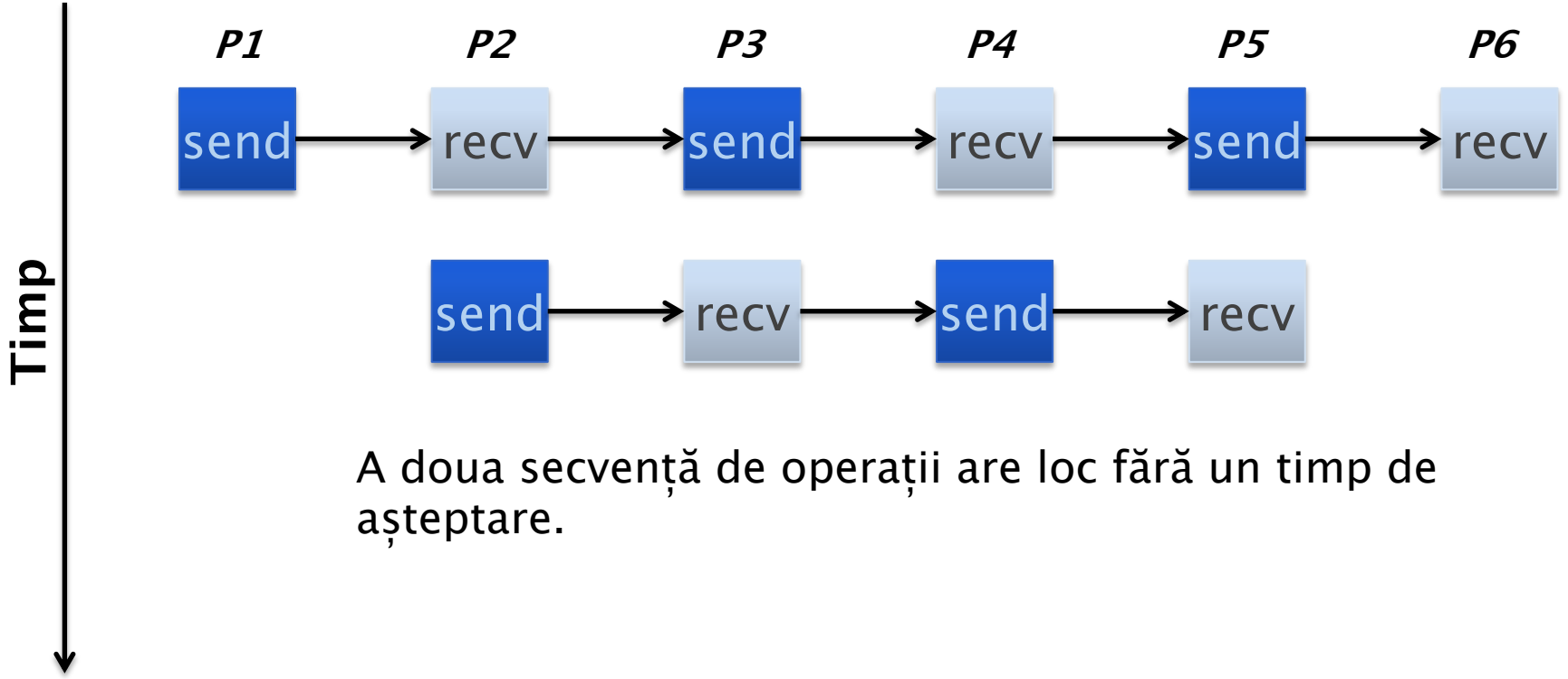
Probleme de transmitere în modul sincron

- Presupunem că:
  - avem 5 procese care execută o serie de *send* și *receive* + 1 proces care execută *receive*
  - inițial, cele 5 procese execută simultan *send* și un proces execută *receive*



Probleme de transmitere în modul sincron

- Soluție:
  - Ordonarea operațiilor *send* și *recv*



A doua secvență de operații are loc fără un timp de așteptare.

Operaţii *send-recv* combinate

```
1   MPI_Sendrecv
2
3   int MPI_Sendrecv(
4       void *sendbuf, int sendcount, MPI_Datatype sendtype,
5       int dest, int sendtag,
6       void *recvbuf, int recvcount, MPI_Datatype recvtype,
7       int source, int recvtag,
8       MPI_Comm comm,
9       MPI_Status *status )
10
11
12  MPI_Sendrecv_replace
13
14  int MPI_Sendrecv_replace(
15      void *buf, int count, MPI_Datatype datatype,
16      int dest, int sendtag, int source, int recvtag,
17      MPI_Comm comm,
18      MPI_Status *status )
```

Transmitere prin tampon alocat explicit

```
1   #define BuffSize 100000
2   int size;
3   char *buff;
4
5   buff = malloc(BuffSize); MPI_Buffer_attach(buff,
    BuffSize);
6   /* Tamponul de BuffSize octeți poate fi folosit acum */
7
8   MPI_Bsend( ... );
9   MPI_Buffer_detach(&buff, &size);
10  /* Tamponul redus la zero */
11
12  MPI_Buffer_attach(buff, size);
13  /* Tamponul disponibil din nou */
14
15
16  int MPI_Buffer_attach(void *buffer, int size );
17  int MPI_Buffer_detach(void *bufferptr, int *size );
```

Transmitere sincronă și în mod pregătit

- Transmitere sincronă
  - MPI_Ssend
- Transmitere în modul pregătit
  - MPI_Rsend

Comunicația non-blocantă

- ```
  int MPI_Isend(void *buf, int count,
                MPI_Datatype datatype, int dest, int tag,
                MPI_Comm comm, MPI_Request *request)
  ```
- ```
  int MPI_Irecv(void *buf, int count,
                MPI_Datatype datatype, int source, int tag,
                MPI_Comm comm, MPI_Request *request)
  ```

- MPI_Test
- MPI_Testall
- MPI_Testany
- MPI_Testsome

- MPI_Wait
- MPI_Waitall
- MPI_Waitany
- MPI_Waitsome

Exemplu

```
1  int rank, size, i, index, a[10], flag, f[6];
2  MPI_Request request, r[6];
3  MPI_Status status, s[6];
4  if (rank == 0) {
5      for (i=0; i<10; i++)
6          a[i] = i+1;
7      MPI_Isend(a, 10, MPI_INT, 1, 0, MPI_COMM_WORLD, &request);
8      MPI_Test(&request, &flag, &status);
9      if (flag) {
10         printf("Procesul 0 ... operatia nu s-a terminat\n");
11     } else {
12         printf("Procesul 0 ... operatia s-a terminat\n");
13     }
14     sleep(10);
15     MPI_Wait(&request, &status);
16     printf("Procesul 0 ... operatia s-a terminat\n");
17 }
18 if (rank == 1) {
19     sleep(15);
20     a[0] = 255;
21     MPI_Irecv(a, 10, MPI_INT, MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &request);
22     MPI_Test(&request, &flag, &status);
23     if (flag) {
24       printf("Procesul 1 ... am primit date");
25     } else {
26       MPI_Wait(&request, &status);
27     }
28 }
```

Evitarea întârzierii nelimitate (1)

```
1   typedef struct {
2       char data [MSIZE];
3       int dsize;
4   } Buffer;
5
6   Buffer buf[];
7   MPI_Request req[];
8   MPI_Status status[];
9   int index[];
10  ...
11  MPI_Comm_rank(comm, &rank);
12  MPI_Comm_size(comm, &size);
13
14  if(rank != size-1) {
15      /* client code */
16      buf = (Buffer *) malloc(sizeof(Buffer));
17      while(1) {
18          produce_request( buf->data, &buf->dsize);
19          MPI_Send(buf->data,buf->dsize,MPI_CHAR,size-1,tag,comm)
20      }
21  }
```

Evitarea întârzierii nelimitate (2)

```
22  else {
23
24      buf = malloc(sizeof(Buffer)*(size-1));
25      req = malloc(sizeof(MPI_Request)*(size-1));
26      status = malloc(sizeof(MPI_Status)*(size-1));
27      index = malloc(sizeof(int)*(size-1));
28
29      for(i=0; i < size-1; i++)
30          MPI_Irecv(buf[i].data,MSIZE,MPI_CHAR,i,tag,comm,&req[i]);
31      while (1){
32          MPI_Waitsome(size-1, req, &count, index, status);
33          for (i=0; i < count; i++){
34              j = index[i];
35              MPI_Get_count(&status[i], MPI_CHAR, &buf[j].dsize);
36              process_request(buf[j].data, buf[j].dsize);
37              MPI_Irecv(buf[j].data,MSIZE,MPI_CHAR,j,tag,comm,&req[j])
38          }
39      }
40  }
```

Cereri de comunicare persistentă

- De multe ori, o comunicație cu *aceeași listă de argumente* este repetată în cadrul unei bucle.
- În astfel de situații, este posibilă optimizarea comunicației prin prin *legarea* listei de argumente la o cerere de comunicare persistentă.
- Cererea de comunicare persistentă reduce timpul de comunicare dintre proces și *controller*-ul de comunicare, nu și timpul de comunicare scurs pe rețeaua externă.
- Nu este necesar ca mesajele trimise prin intermediul cererilor persistente să fie recepționate în mod persistent (sau viceversa).

Cereri de comunicare persistentă

- O cerere de comunicare persistentă este inactivă după ce a fost creată.
- O comunicare (operație *send* sau *receive*) care folosește o cerere persistentă trebuie inițiată (cu ajutorul funcției *MPI_Start*).
- Astfel, programatorul poate să:
  - creeze o cerere persistentă – *MPI_Send_init*, *MPI_Recv_init*
  - declanșeze o operație – *MPI_Start*, *MPI_Startall*
  - aștepte terminarea unei operații – *MPI_Wait*
  - elimine înregistrarea cererii persistente – *MPI_Request_free*

Cereri de comunicare persistentă

- Sintaxă:
  - `int MPI_Start(MPI_Request *request)`
  - `int MPI_Wait(MPI_Request *request, MPI_Status *status)`
  - `int MPI_Request_free(MPI_Request *request )`
- Exemplu:

```
MPI_request *request;
MPI_Recv_init(buf, count, datatype,
              source, tag, comm, request);

/* MPI_Start are semantica MPI_Irecv */
MPI_Start(request);
MPI_Wait(request, status);
MPI_Request_free(request);
```

Tipuri de date predefinite în MPI

- **Tipuri predefinite**
  - toate tipurile tipurile de bază din C *(și din FORTRAN)* plus *MPI_BYTE* și *MPI_PACKED*.

Tip MPI	Tip echivalent în C
MPI_CHAR MPI_SHORT MPI_INT	signed char signed short int signed int
MPI_LONG MPI_UNSIGNED_CHAR MPI_UNSIGNED_SHORT	signed long int unsigned char unsigned short int
MPI_UNSIGNED_INT MPI_UNSIGNED_LONG	unsigned int unsigned long int
MPI_DOUBLE MPI_LONG_DOUBLE	double long double

Tipuri de date derivate în MPI

- Motivație: realizarea unei modalități eficiente de a comunica tipuri *non-contigue* sau *diferite* într-un mesaj
- Formal, standardul MPI definește tipul de date general alcătuit din:
  - o secvență de tipuri de bază
  - o secvență de deplasări (deplasările au o valoare întreagă)
- Harta unui tip (*typemap*) este folosită pentru o exprimare facilă:
  - Harta tipului = secvență de perechi (tip, deplasare)
  - $\boldsymbol{Typemap} = \{(\boldsymbol{type_0}, \boldsymbol{displacement_0}), \ \ldots, (\boldsymbol{type_{n-1}}, \boldsymbol{displacement_{n-1}})\}$
- Semnătura tipului (*typesig*) = secvența tipurilor
  - $\boldsymbol{Typesig} = \{\boldsymbol{type_0}, \boldsymbol{type_1}, \ldots, \boldsymbol{type_{n-1}}\}$
- Titlul tipului *(handle)*
  - Exemplu: MPI_INT, cu harta $Typemap = \{(int, 0)\}$

Tipuri de date derivate în MPI

- Etapele necesare pentru a utiliza un tip derivat:
  - construirea tipului (utilizând o serie de constructori speciali)
  - alocarea sa (înregistrarea folosind *MPI_Type_commit*)
  - la final, tipul va fi dezalocat (*MPI_Type_free*)

- Exemple de constructori:
  - **MPI_Type_contiguous** (count, oldtype, *newtype)
  - **MPI_Type_vector** (count, blocklength, stride, oldtype, *newtype)
  - **MPI_Type_indexed** (count, blocklens[], offsets[], old_type, *newtype)
  - **MPI_Type_struct** (count, blocklens[], offsets[], old_types[], *newtype)

Tipuri de date derivate în MPI - constructori

- **MPI_Type_contiguous** (count, oldtype, *newtype)
  - Se produce un nou tip de date prin copierea *oldtype* de *count* ori

- Exemplu:

```
count = 4;
MPI_Type_contiguous(count, MPI_FLOAT, &rowtype);

MPI_Send(&a[2][0], 1, rowtype, dest, tag, comm);
```

Tipuri de date derivate în MPI - constructori

- **MPI_Type_vector** (count, blocklength, stride, oldtype, *newtype)
  - Se vor lua *count* blocuri de lungime *blocklength*
  - Un element din bloc este de tipul *oldtype*
  - Blocurile pot fi despărțite de zone care vor fi sărite (*strides*)

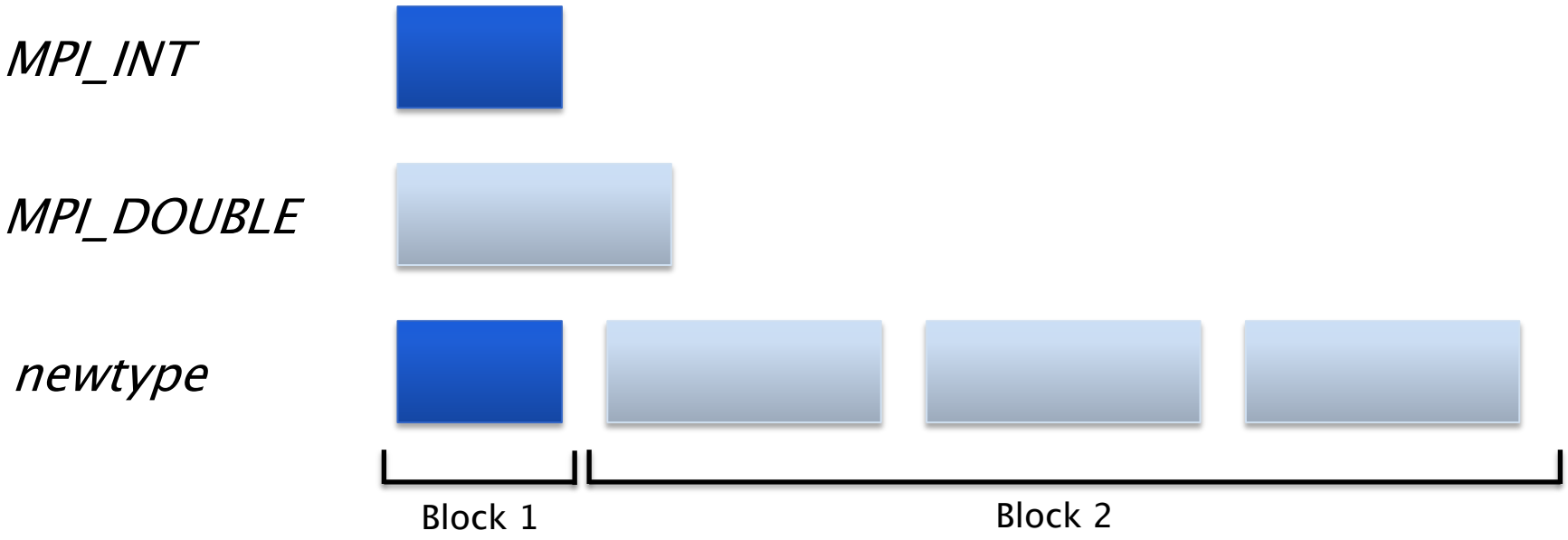
- Exemplu:

```
float mesh[10][20];
int dest, tag;
int count = 10;                 /* numărul de blocuri */
int blocklen = 1;               /* o singură coloană */
int stride = 19;                /* salt de 20 de elemente */

MPI_Datatype newtype;
MPI_Type_vector(count,blocklen,stride,MPI_FLOAT,&newtype);
MPI_Type_commit(&newtype);
MPI_Send(&mesh[0][19], 1, newtype, dest, tag, MPI_COMM_WORLD);
```

Tipuri de date derivate în MPI - constructori

- **MPI_Type_struct** (count, blocklens, offsets, oldtypes, *newtypes)
  - Se vor lua *count* blocuri de lungime *blocklength*
  - Un element din bloc este de tipul *oldtype*
  - Fiecare bloc se găsește la deplasamentul corespunzător din *offsets*
- Exemplu: un apel **MPI_Type_struct** cu *count*=2, *blocklens[0]*=1, *old_types[0]*=MPI_INT, *blocklens[1]*=3 şi *old_types[1]*=MPI_DOUBLE



MPI_type_struct - exemplu

```
1  # define MAX_PART 1000
2  Struct Part_struct
3    { int class;       /* clasa particulei */
4      double d[6];     /* coordonatele particulei */
5      char b[7];       /* alte informații */
6    };
7  struct Part_struct particle[MAX_PART];
   
8  /* construcția tipului care descrie structura MPI a unei particule */
9  MPI_Datatype PartType;
10 MPI_Datatype type[3] = {MPI_INT, MPI_DOUBLE, MPI_CHAR}; /* semnătura */
11 int blocklen [3] = { 1, 6, 7};
12 MPI_Aint disp[3];
13 int base;
```

MPI_type_struct - exemplu

```
14  /* se determină adresele absolute pentru a calcula ulterior deplasările
15   * MPI_Address este folosit pentru portabilitate
16   */
17  MPI_Address (particle, disp);
18  MPI_Address (particle[0].d, disp+1);
19  MPI_Address (particle[0].b, disp+2);
20
21  base=disp[0];
22  /* deplasările reprezintă adresele relative */
23  for (i=0; i<3; i++) disp [i] -= base;
24  MPI_Type_create_struct ( 3, blocklen, disp, type, &PartType);
25  MPI_Type_commit (&PartType);
26
27  /* se transmite întregul tablou */
28  MPI_Send (particle, MAX_PART, PartType, dest, tag, comm);
```

Comunicare colectivă

- Comunicarea colectivă se referă la acele funcții din MPI care implică **toate procesele** din comunicator.
- Caracteristici:
  - O operație colectivă presupune apelul funcției corespunzătoare de către toate procesele din comunicator, folosind argumente potrivite.
  - Dimensiunea datelor trimise trebuie să **coincidă** cu dimensiunea datelor primite.
  - Operațiile colective se pot executa doar în **modul blocant**.
- Operațiile de comunicare colectivă se împart în:
  - Operații de sincronizare
  - Operații de transfer de date
  - Calcule colective (de reducere)

Operații de sincronizare
Sincronizarea cu barieră

- `int MPI_Barrier(MPI_Comm comm)`

- MPI_Barrier sincronizează toate procesele din comunicator
- Un proces apelant se blochează până când toate celelalte procese din comunicator apelează, la rândul lor, această funcție

Operații de transfer de date

- Operații de transfer de date:
  - **Difuzarea** (*broadcast*)
    - propagarea unui mesaj de la *rădăcină* la restul proceselor
  - **Colectarea** (*gather*)
    - procesul *rădăcină* colectează date de la celelalte procese
  - **Distribuirea** (*scatter*)
    - inversa operației de colectare
  - Variații ale colectării și distribuirii (*allgather*, *alltoall*)
    - *allgather*
      - fiecare proces colectează datele de la celelalte procese
    - *alltoall*
      - fiecare proces realizează o distribuire a datelor, în funcție de rangul celorlalte procese

Operații de transfer de date

Difuzare

P_0	A			
P_1	A			
P_2	A			
P_3	A			

Distribuire

P_0	A	B	C	D
P_1	B			
P_2	C			
P_3	D			

Colectare

P_0	A	B	C	D
P_1	B			
P_2	C			
P_3	D			

Operații de transfer de date

Allgather

P_0	A	B	C	D
P_1	B	A	C	D
P_2	C	A	B	D
P_3	D	A	B	C

Alltoall

P_0	A_0	B_0	C_0	D_0
P_1	A_1	B_1	C_1	D_1
P_2	A_2	B_2	C_2	D_2
P_3	A_3	B_3	C_3	D_3

Calcule colective (operații de reducere)

- `int MPI_Reduce(void *sendbuf, void *recvbuf, int count, MPI_Datatype datatype, MPI_Op op, int root, MPI_Comm comm)`

P_0 *sendbuf*	A_0	A_1	A_2	A_3
P_1 *sendbuf*	B_0	B_1	B_2	B_3
P_2 *sendbuf*	C_0	C_1	C_2	C_3
P_3 *sendbuf*	D_0	D_1	D_2	D_3

MPI_Op = MPI_SUM

Rezultat MPI_Reduce: P_0 *recvbuf* va conține suma celor 4 vectori

P_0	$A_0 + B_0 + C_0 + D_0$	$A_1 + B_1 + C_1 + D_1$	$A_2 + B_2 + C_2 + D_2$	$A_3 + B_3 + C_3 + D_3$
P_1				
P_2				
P_3				

Topologii virtuale

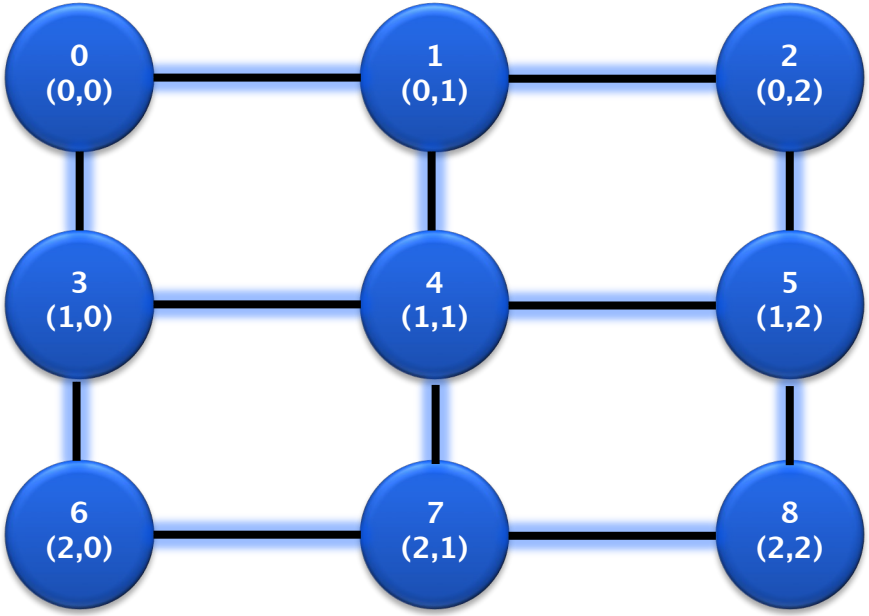
- O topologie este un mecanism prin care se obține o modalitate facilă de a *numi* procesele dintr-un grup (comunicator).
- În cazul multor aplicații distribuite, simpla folosire a rangurilor nu reflectă modelul logic de comunicare între procese.
- Procesele pot fi văzute ca făcând parte dintr-un:
  - grid sau structură carteziană (două/trei/... dimensiuni)
  - graf (model generic)
- Trebuie făcută distincția între:
  - topologia virtuală
  - topologia hardware-ului
- Topologia virtuală poate fi concepută astfel încât să respecte modul în care procesele sunt repartizate procesoarelor (topologia hardware), pentru o sporire a performanței comunicării dintre procese.

Topologii virtuale

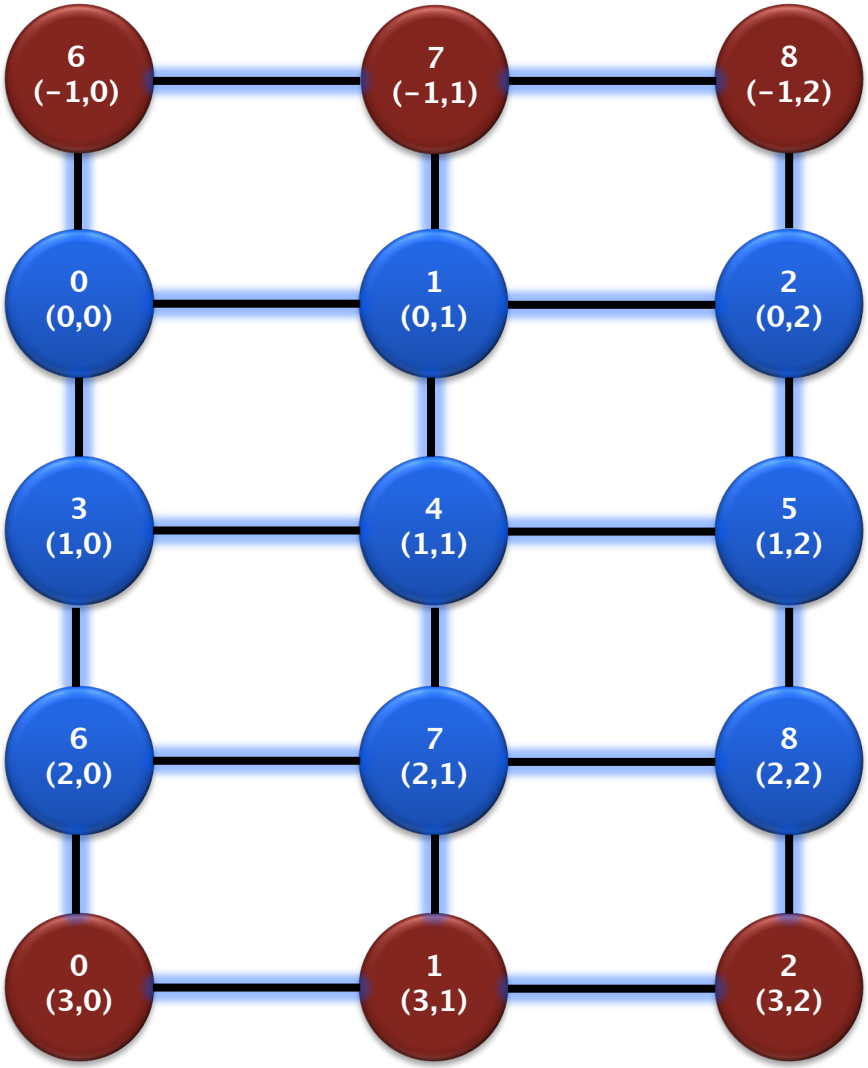
- `MPI_Cart_create (MPI_Comm comm_old, int ndims, int *dims, int *periods, int reorder, MPI_Comm comm_cart)`
- Funcția creează un nou comunicator în care topologia este una carteziană, conform parametrilor (procesele implicate sunt cele din *comm_old*):
  - *ndims* – numărul de dimensiuni din structura carteziană
  - *dims* – un vector de dimensiune *ndims* care specifică numărul de procese de pe fiecare dimensiune
  - *periods* – un vector cu valori de tip boolean care indică prezența/absența periodicității pe o anumită dimensiune
  - *reorder* – valoare booleană care permite (sau nu) MPI-ului să reordoneze procesele din noul comunicator în funcție de proximitățile fizice ale procesoarelor pe care rulează procesele (topologia hardware)

Exemple de grile carteziene 2D
Grilă neperiodică

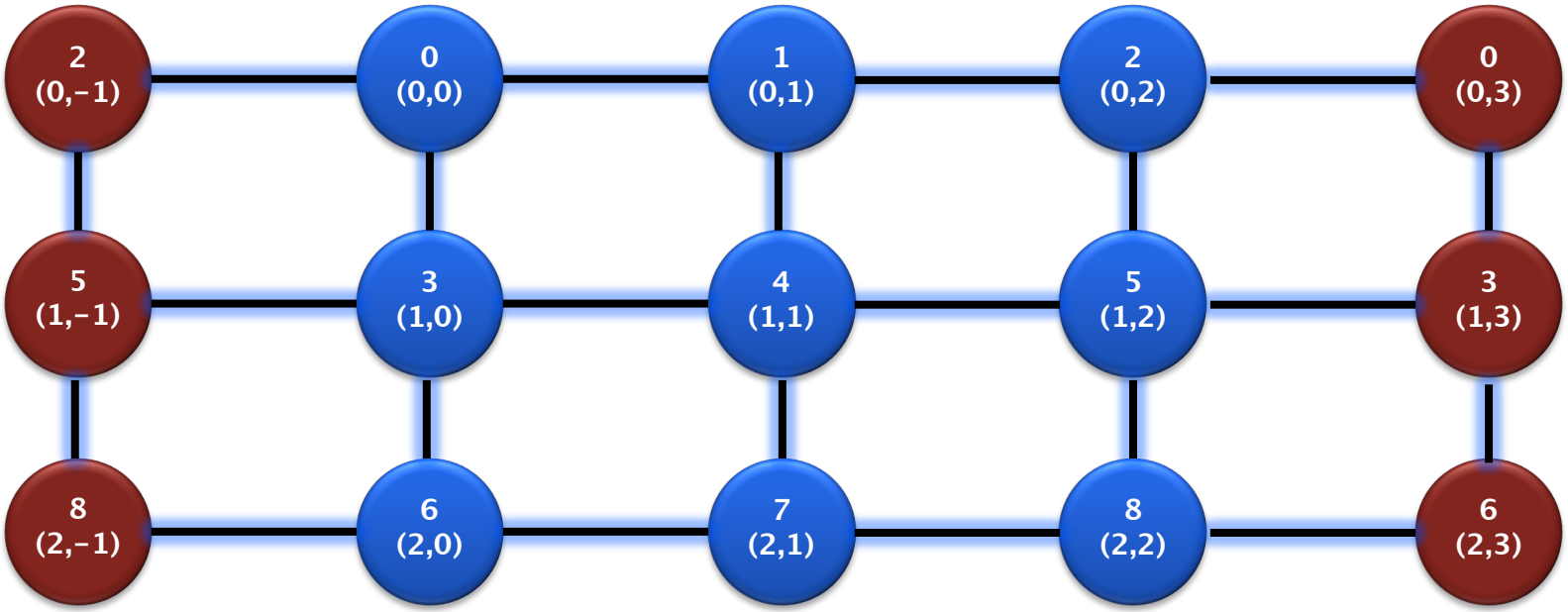
Rang
(linie,coloană)



Exemple de grile carteziene 2D
Grilă periodică pe linii



Exemple de grile carteziene 2D
Grilă periodică pe coloane



Primitive pentru topologii carteziene

- Furnizare *dims*, *periods* și *coords* pentru procesul apelant:

```
int MPI_Cart_get(MPI_Comm comm, int maxdims,
                 int *dims, int *periods,
                 int *coords)
```

- Aflare *coords* pentru un proces cu rang cunoscut:

```
int MPI_Cart_coords(MPI_Comm comm, int rank,
                    int maxdims, int *coords)
```

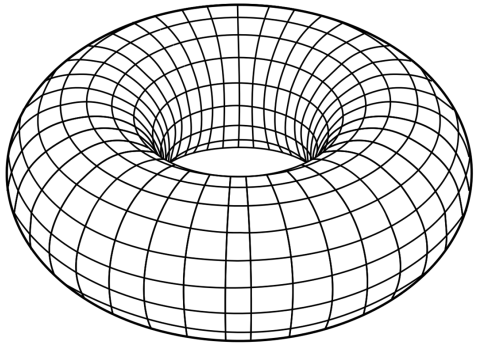
- Pentru a determina sursa și destinația care vor fi folosite într-un apel ulterior *MPI_Sendrecv* (sursa și destinația pe aceeași direcție):

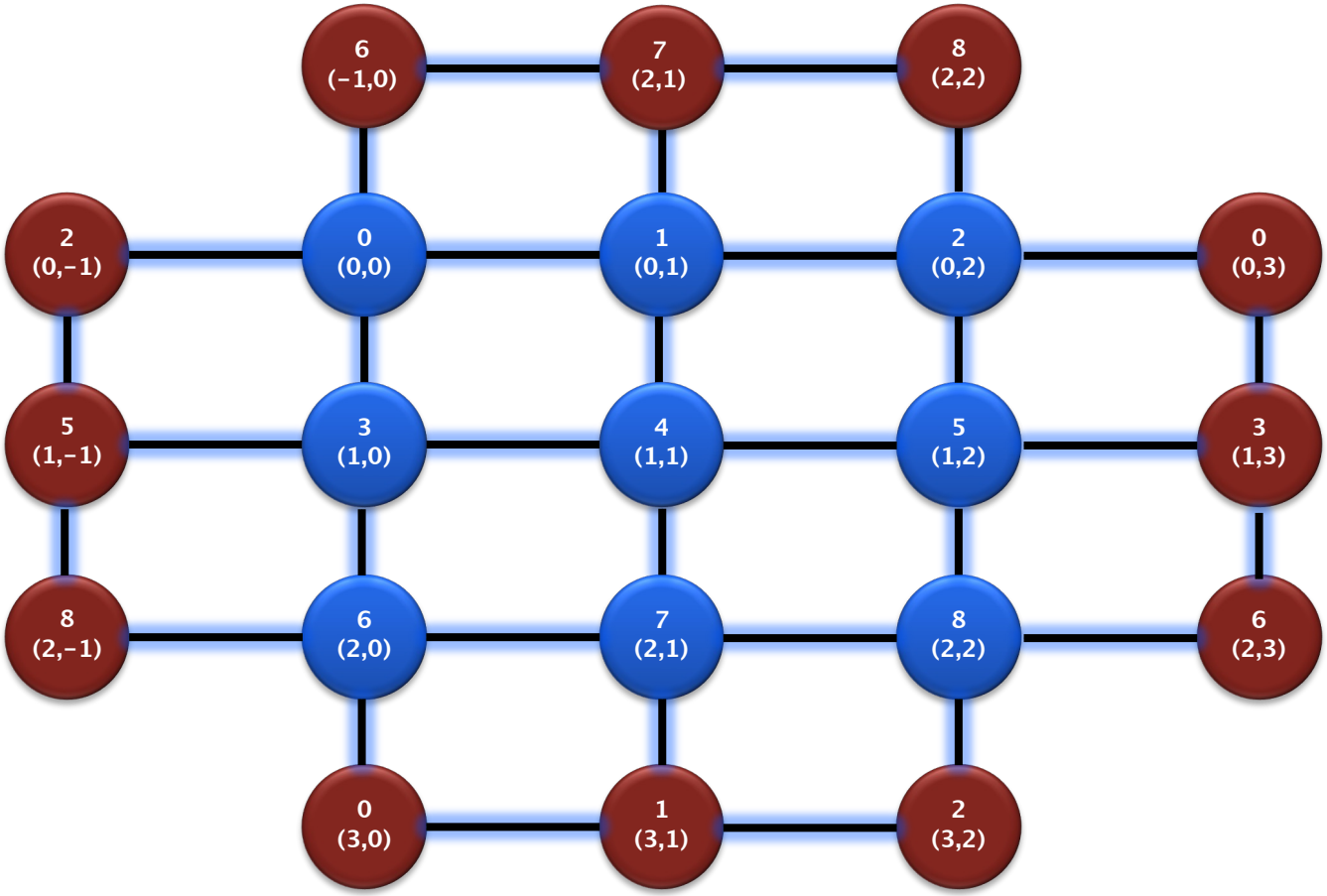
```
int MPI_Cart_shift(MPI_Comm comm, int direction,
                   int displ, int *source, int *dest)
```

Obs: sursa = rang proces apelant – deplasament

destinația = rang proces apelant + deplasament

Grilă periodică pe toate dimensiunile (tor)
MPI_Cart_shift





Proces apelant: **5**
Direcție: 1 (linia comună)
Deplasament: 2
Sursa: **3**
Destinația: **4**

Exemplu (1)

```
1  int numtasks, rank, newrank, i, j;
2  float a[SIZE][SIZE] = {1.0, 2.0, 3.0, 4.0}, { 5.0, 6.0, 7.0, 8.0 },
3                        { 9.0, 10.0, 11.0, 12.0 }, { 13.0, 14.0, 15.0, 16.0} };
4  float t;
5  MPI_Status stat;
6  MPI_Request request;
7  MPI_Comm newcomm;
8  int ndims, reorder, dim_size[2], periods[2];
9  int coords[2], source, dest;
10 MPI_Init(&argc,&argv);
11 MPI_Comm_rank(MPI_COMM_WORLD, &rank);
12 MPI_Comm_size(MPI_COMM_WORLD, &numtasks);
13 ndims = 2; /* numărul de dimensiuni */
14 dim_size[0] = 4; /* numărul de procese pe prima dimensiune */
15 dim_size[1] = 4; /* numărul de procese pe cea de-a doua dimensiune */
16 periods[0] = 1; /* dimensiune ciclică */
17 periods[1] = 0;
18 reorder = 1;
19 MPI_Cart_create(MPI_COMM_WORLD, ndims, dim_size, periods, reorder, &newcomm);
20 MPI_Comm_rank(newcomm, &newrank);
21 MPI_Cart_coords(newcomm, newrank, 2, coords);
22 printf("Rang=%d Pozitia=(%d,%d)\n", newrank, coords[0], coords[1]);
23 MPI_Barrier(MPI_COMM_WORLD);
24 MPI_Cart_shift(newcomm, 0, 1, &source, &dest);
25 printf("Procesul %d: trimit catre %d si primesc de la %d\n", newrank, dest, source);
```

Exemplu (2)

```
26  if (coords[0] % 2 == 0) {
27      MPI_Send(&a[coords[0]][coords[1]], 1, MPI_FLOAT, dest, 0, newcomm);
28      MPI_Recv(&t, 1, MPI_FLOAT, source, 0, newcomm, &stat);
29  } else {
30      MPI_Recv(&t, 1, MPI_FLOAT, source, 0, newcomm, &stat);
31      MPI_Send(&a[coords[0]][coords[1]], 1, MPI_FLOAT, dest, 0, newcomm);
32  }
33  /* se pune noul element pe poziția corespunzătoare */
34  a[coords[0]][coords[1]] = t;
35  MPI_Barrier(MPI_COMM_WORLD);
36  if (newrank == 0) {
37      for (i=0; i<4; i++) {
38          for (j=0; j<4; j++) {
39              coords[0] = i; coords[1] = j;
40              MPI_Cart_rank(newcomm, coords, &rank);
41              if (rank != 0) {
42                  MPI_Recv(&a[j][i], 1, MPI_FLOAT, rank, 0, newcomm, &stat);
43              }
44              printf("%.2f ", a[j][i]);
45          }
46          printf("\n");
47      }
48  } else {
49      MPI_Send(&a[coords[0]][coords[1]], 1, MPI_FLOAT, 0, 0, newcomm);
50  }
51  MPI_Finalize();
```

Sumar

- *Message Passing Interface*
  - Elemente introduse
  - Comunicarea punct – la – punct
  - Tipuri de date
  - Topologii de procese
  - Comunicare colectivă